

MÓDULO DE TESTING

Pruebas Unitarias — Entrega 05

Ingeniería de Software 1 (2016701)

Universidad Nacional de Colombia — Sede Bogotá

Proyecto	Bolirana — Plataforma de apuestas deportivas
Backend	Spring Boot 3.3 · Java 17
Framework de tests	JUnit 5 + Mockito + AssertJ
Coverage	JaCoCo 0.8.12
Fecha	Junio 2025

1. Introducción

Este documento presenta las pruebas unitarias implementadas para el backend de Bolirana, una plataforma de apuestas deportivas desarrollada con Spring Boot. Las pruebas validan la lógica de negocio real de la capa de servicios, sin acceder a la base de datos, utilizando mocks (Mockito) para aislar las dependencias externas.

⚠ Nota: Las pruebas sobre escrituras/lecturas directas a BD son pruebas de integración y están fuera del alcance de este módulo, conforme al enunciado.

Herramientas utilizadas

Herramienta	Versión	Propósito
JUnit 5 (Jupiter)	vía Spring Boot 3.3	Framework de pruebas unitarias
Mockito	vía spring-boot-starter-test	Mocking de repositorios JPA
AssertJ	vía spring-boot-starter-test	Aserciones fluidas y expresivas
JaCoCo	0.8.12	Reporte de cobertura de código

2. Estructura del proyecto de pruebas

Los tests se ubican en el paquete estándar de Maven para pruebas, siguiendo la convención `NombreServiceTest.java`:

```
src/test/java/com/bolirana/backend/service/
├── EventoServiceTest.java           ← Test 1 y 2 (Integrante A)
├── MovimientoSaldoServiceTest.java  ← Test 3 (Integrante A)
├── ApuestaServiceTest.java         ← Tests 4,5,6 (Integrante B)
├── ConfiguracionRiesgoServiceTest.java ← Tests 7,8,9 (Integrante C)
└── UsuarioServiceTest.java         ← Tests 10,11,12 (Integrante D)

# Ejecutar todos los tests + reporte de cobertura:
./mvnw test jacoco:report

# Reporte HTML: target/site/jacoco/index.html
```

3. Pruebas unitarias por integrante

3.1 Juan Pablo Zuluaga — EventoService & MovimientoSaldoService

Se implementaron 3 pruebas unitarias cubriendo dos servicios. Todas usan `@ExtendWith(MockitoExtension.class)`, `@Mock` / `@InjectMocks` y `@DisplayName` descriptivo en español.

Resumen de pruebas

#	Método	Escenario	Caso límite	Esperado
1	buscarPorId(Long)	ID válido (id=1L) presente	Frontera: entidad existe	Optional con datos correctos ✓
2	buscarPorId(Long)	ID inexistente (id=999L)	Frontera: ausente → 404	Optional vacío ✓

#	Método	Escenario	Caso límite	Esperado
3	listarPorUsuario(Long)	Usuario con 2 movimientos	Filtrado correcto por usuarioId	Lista exacta de 2 elementos 

EventoServiceTest.java

```
@ExtendWith(MockitoExtension.class)
class EventoServiceTest {

    @Mock private EventoRepository eventoRepository;
    @InjectMocks private EventoService eventoService;

    @Test
    @DisplayName("buscarPorId retorna el evento cuando existe en el repositorio")
    void buscarPorId_eventoExistente_retornaEvento() {
        // Caso límite: el id corresponde a un evento presente en el repositorio
        Long id = 1L;
        Evento evento = new Evento();
        evento.setId(id); evento.setNombre("Final Copa America");
        evento.setDeporte("Futbol"); evento.setEstado("ABIERTO");

        when(eventoRepository.findById(id)).thenReturn(Optional.of(evento));
        Optional<Evento> resultado = eventoService.buscarPorId(id);

        assertThat(resultado).isPresent();
        assertThat(resultado.get()).isEqualTo(evento);
        verify(eventoRepository).findById(id);
    }

    @Test
    @DisplayName("buscarPorId retorna Optional vacío cuando el evento no existe")
    void buscarPorId_eventoInexistente_retornaOptionalVacio() {
        // Caso límite: id=999L no existe → controller debe responder 404
        Long id = 999L;
        when(eventoRepository.findById(id)).thenReturn(Optional.empty());

        Optional<Evento> resultado = eventoService.buscarPorId(id);

        assertThat(resultado).isEmpty();
        verify(eventoRepository).findById(id);
    }
}
```

MovimientoSaldoServiceTest.java

```
@ExtendWith(MockitoExtension.class)
class MovimientoSaldoServiceTest {

    @Mock private MovimientoSaldoRepository movimientoSaldoRepository;
    @InjectMocks private MovimientoSaldoService movimientoSaldoService;

    @Test
    @DisplayName("listarPorUsuario retorna la lista filtrada por usuarioId")
    void listarPorUsuario_usuarioConMovimientos_retornaListaFiltrada() {
        // Caso límite: solo devuelve movimientos del usuarioId solicitado
        Long usuarioId = 5L;
        Usuario usuario = new Usuario(); usuario.setId(usuarioId);

        MovimientoSaldo deposito = new MovimientoSaldo(); deposito.setTipo("DEPOSITO");
        deposito.setUsuario(usuario); deposito.setMonto(100000.0);
        MovimientoSaldo retiro = new MovimientoSaldo(); retiro.setTipo("RETIRO");
        retiro.setUsuario(usuario); retiro.setMonto(50000.0);

        when(movimientoSaldoRepository.findByUsuarioId(usuarioId))
            .thenReturn(List.of(deposito, retiro));

        List<MovimientoSaldo> resultado =
```

```
movimientoSaldoService.listarPorUsuario(usuarioId);

assertThat(resultado).hasSize(2).containsExactly(deposito, retiro)
    .allSatisfy(m -> assertThat(m.getUsuario().getId()).isEqualTo(usuarioId));
verify(movimientoSaldoRepository).findByUsuarioId(usuarioId);
}
```

Resultados de ejecución (BUILD SUCCESS)

Clase de test	Tests	Pasaron	Fallaron	Estado
EventoServiceTest	2	2	0	✓ PASS
MovimientoSaldoServiceTest	1	1	0	✓ PASS
TOTAL	3	3	0	✓ BUILD SUCCESS

3.2 Samuel Ricardo Pardo Trujillo — ApuestaService

Se implementaron 3 pruebas unitarias cubriendo el servicio de registro de apuestas. Todas usan `@ExtendWith(MockitoExtension.class)`, `@Mock` / `@InjectMocks` y `@DisplayName` descriptivo en español.

Resumen de pruebas

#	Método	Escenario	Caso límite	Esperado
1	crear()	Opción existente con cuota 2.5	La cuota puede cambiar después; debe guardarse el valor vigente al momento de crear	cuotaCongelada = 2.5 
2	crear()	Opción existente, estado no enviado	El estado inicial siempre debe ser PENDIENTE sin importar lo que el cliente envíe	estado = "PENDIENTE" 
3	crear()	opcionId = 999L no existe	No debe persistirse ninguna apuesta con una opción inválida	IllegalArgumentException, save() nunca llamado 

ApuestaServiceTest.java

```
@ExtendWith(MockitoExtension.class)
class ApuestaServiceTest {

    @Mock
    private ApuestaRepository apuestaRepository;

    @Mock
    private OpcionApuestaRepository opcionApuestaRepository;

    @InjectMocks
    private ApuestaService apuestaService;

    @Test
    @DisplayName("crear() asigna cuotaCongelada igual a cuotaActual de la opción en el momento de registrar")
    void crear_opcionExistente_congelajCuotaActual() {
        // Caso límite: la cuota puede cambiar después; lo que se guarda debe ser el valor vigente al momento de crear
        OpcionApuesta opcion = new OpcionApuesta();
        opcion.setId(1L);
        opcion.setCuotaActual(2.5);

        Apuesta apuesta = new Apuesta();
```

```

        apuesta.setOpcion(opcion);
        apuesta.setMonto(10000.0);

        when(opcionApuestaRepository.findById(1L)).thenReturn(Optional.of(opcion));
        when(apuestaRepository.save(apuesta)).thenReturn(apuesta);

        Apuesta resultado = apuestaService.crear(apuesta);

        assertThat(resultado.getCuotaCongelada()).isEqualTo(2.5);
    }

    @Test
    @DisplayName("crear() inicializa el estado de la apuesta en PENDIENTE")
    void crear_opcionExistente_inicializaEstadoPendiente() {
        // Caso límite: el estado inicial siempre debe ser PENDIENTE sin importar lo
        // que el cliente envíe
        OpcionApuesta opcion = new OpcionApuesta();
        opcion.setId(2L);
        opcion.setCuotaActual(1.8);

        Apuesta apuesta = new Apuesta();
        apuesta.setOpcion(opcion);
        apuesta.setMonto(5000.0);

        when(opcionApuestaRepository.findById(2L)).thenReturn(Optional.of(opcion));
        when(apuestaRepository.save(apuesta)).thenReturn(apuesta);

        Apuesta resultado = apuestaService.crear(apuesta);

        assertThat(resultado.getEstado()).isEqualTo("PENDIENTE");
    }

    @Test
    @DisplayName("crear() lanza IllegalArgumentException y no llama a save() cuando la opción no existe")
    void crear_opcionInexistente_lanzaExcepcionSinGuardar() {
        // Caso límite: el id de opción no existe en la base de datos, la apuesta no
        // debe persistirse
        OpcionApuesta opcionInexistente = new OpcionApuesta();
        opcionInexistente.setId(999L);

        Apuesta apuesta = new Apuesta();
        apuesta.setOpcion(opcionInexistente);
        apuesta.setMonto(10000.0);

        when(opcionApuestaRepository.findById(999L)).thenReturn(Optional.empty());

```

```

    assertThatThrownBy(() -> apuestaService.crear(apuesta))
        .isInstanceOf(IllegalArgumentException.class)
        .hasMessage("Opcion de apuesta no encontrada");

    verify(apuestaRepository, never()).save(apuesta);
}
}

```

Resultados de ejecución (BUILD SUCCESS)

Clase de test	Tests	Pasaron	Fallaron	Estado
ApuestaServiceTest	3	3	0	✓ PASS
TOTAL	3	3	0	✓ BUILD SUCCESS

3.3 [Camilo Andrés Salinas Cuervo] — ConfiguracionRiesgoService

Se implementaron 3 pruebas unitarias cubriendo el servicio de configuración de riesgo. Todas usan `@ExtendWith(MockitoExtension.class)`, `@Mock` / `@InjectMocks` y `@DisplayName` descriptivo en español.

Resumen de pruebas

#	Método	Escenario	Caso límite	Esperado
1	crearOActualizar()	Config existente para el mercado	Solo debe modificarse limiteAlerta sin crear registro nuevo	limiteAlerta actualizado a 750.000 ✓
2	crearOActualizar()	No existe config previa	La config entrante debe persistirse como nuevo registro	Objeto entrante guardado directamente ✓
3	crearOActualizar()	Verificación de integridad del mercadold	El argumento de save() no debe alterar el mercadold original	mercadold = 1L confirmado con argThat() ✓

ConfiguracionRiesgoServiceTest.java

```

@ExtendWith(MockitoExtension.class)
class ConfiguracionRiesgoServiceTest {

    @Mock
    private ConfiguracionRiesgoRepository configuracionRiesgoRepository;

    @InjectMocks
    private ConfiguracionRiesgoService configuracionRiesgoService;
}

```

```

@Test
@DisplayName("crearOActualizar() actualiza limiteAlerta cuando ya existe config
para el mercado")
void crearOActualizar_configExistente_actualizaLimiteAlerta() {
    // Caso límite: el mercado ya tiene config, por lo que solo debe modificarse
    limiteAlerta sin crear un nuevo registro

    Mercado mercado = new Mercado();
    mercado.setId(1L);

    ConfiguracionRiesgo existente = new ConfiguracionRiesgo();
    existente.setId(10L);
    existente.setMercado(mercado);
    existente.setLimiteAlerta(500000.0);

    ConfiguracionRiesgo entrante = new ConfiguracionRiesgo();
    entrante.setMercado(mercado);
    entrante.setLimiteAlerta(750000.0);

    when(configuracionRiesgoRepository.findByMercadoId(1L)).thenReturn(Optional.of(exis
tente));
    when(configuracionRiesgoRepository.save(existente)).thenReturn(existente);

    ConfiguracionRiesgo resultado =
configuracionRiesgoService.crearOActualizar(entrante);

    assertThat(resultado.getLimiteAlerta()).isEqualTo(750000.0);
    verify(configuracionRiesgoRepository, times(1)).save(existente);
}

@Test
@DisplayName("crearOActualizar() crea registro nuevo cuando no existe config
para el mercado")
void crearOActualizar_configInexistente_creaRegistroNuevo() {
    // Caso límite: el mercado no tiene config previa, por lo que la
    configuración entrante debe persistirse como nuevo registro

    Mercado mercado = new Mercado();
    mercado.setId(1L);

    ConfiguracionRiesgo entrante = new ConfiguracionRiesgo();
    entrante.setMercado(mercado);
    entrante.setLimiteAlerta(750000.0);

    when(configuracionRiesgoRepository.findByMercadoId(1L)).thenReturn(Optional.empty()
);

```

```

        when(configuracionRiesgoRepository.save(entrante)).thenReturn(entrante);

        ConfiguracionRiesgo resultado =
configuracionRiesgoService.crearOActualizar(entrante);

        assertThat(resultado).isSameAs(entrante);
        verify(configuracionRiesgoRepository, times(1)).save(entrante);
    }

    @Test
    @DisplayName("crearOActualizar() guarda un objeto cuyo mercadoId es el correcto (1L)")
    void crearOActualizar_objetoGuardado_tieneMercadoIdCorrecto() {
        // Caso límite: se inspecciona el argumento real de save() para garantizar
        // que el mercadoId no fue alterado por el servicio
        Mercado mercado = new Mercado();
        mercado.setId(1L);

        ConfiguracionRiesgo entrante = new ConfiguracionRiesgo();
        entrante.setMercado(mercado);
        entrante.setLimiteAlerta(300000.0);

        when(configuracionRiesgoRepository.findById(1L)).thenReturn(Optional.empty());

        when(configuracionRiesgoRepository.save(entrante)).thenReturn(entrante);

        configuracionRiesgoService.crearOActualizar(entrante);

        verify(configuracionRiesgoRepository).save(
            argThat(config -> config.getMercado().getId().equals(1L))
        );
    }
}

```

Resultados de ejecución (BUILD SUCCESS)

Clase de test	Tests	Pasaron	Fallaron	Estado
ConfiguracionRiesgoServiceTest	3	3	0	✅ PASS
TOTAL	3	3	0	✅ BUILD SUCCESS

3.4 [Daniel Santiago Rincón Santofimio — UsuarioService

Tests asignados: *UsuarioService* — búsqueda por ID (existe / no existe) y creación de usuario.

#	Método	Escenario	Caso límite	Esperado
1	buscarPorId(Long)	ID válido (id=1L) presente	Frontera: entidad existe en el repositorio	Optional con datos correctos ✓
2	buscarPorId(Long)	ID inexistente (id=999L)	Frontera: ausente → 404	Optional.empty() ✓
3	crear(Usuario)	Registro de nuevo usuario	Inserción: persistencia de datos correctos	Usuario guardado con éxito y verificado en repositorio ✓

UsuarioServiceTest.java

```
@ExtendWith(MockitoExtension.class)
class UsuarioServiceTest {

    @Mock
    private UsuarioRepository usuarioRepository;

    @InjectMocks
    private UsuarioService usuarioService;

    @Test
    @DisplayName("buscarPorId retorna el usuario cuando existe en el repositorio")
    void buscarPorId_usuarioExistente_retornaUsuario() {

        Long id = 1L;
        Usuario usuario = new Usuario();
        usuario.setId(id);

        when(usuarioRepository.findById(id)).thenReturn(Optional.of(usuario));

        Optional<Usuario> resultado = usuarioService.buscarPorId(id);

        assertThat(resultado).isPresent();
        assertThat(resultado.get()).isEqualTo(usuario);
        verify(usuarioRepository).findById(id);
    }

    @Test
    @DisplayName("buscarPorId retorna Optional vacio cuando el usuario no existe")
```

```

void buscarPorId_usuarioInexistente_retornaOptionalVacio() {

    Long id = 999L;

    when(usuarioRepository.findById(id)).thenReturn(Optional.empty());

    Optional<Usuario> resultado = usuarioService.buscarPorId(id);

    assertThat(resultado).isEmpty();
    verify(usuarioRepository).findById(id);
}

@Test
@DisplayName("crear guarda el usuario de forma correcta")
void crear_usuarioValido_guardaYRetornaUsuario() {

    Usuario nuevoUsuario = new Usuario();

    when(usuarioRepository.save(nuevoUsuario)).thenReturn(nuevoUsuario);

    Usuario resultado = usuarioService.crear(nuevoUsuario);

    assertThat(resultado).isNotNull();
    assertThat(resultado).isEqualTo(nuevoUsuario);
    verify(usuarioRepository).save(nuevoUsuario);
}
}

```

Clase de test	Tests	Pasaron	Fallaron	Estado
UsuarioServiceTest	3	3	0	✅ PASS
TOTAL	3	3	0	✅ BUILD SUCCESS

4. Cobertura de código — JaCoCo

JaCoCo 0.8.12 fue configurado en pom.xml mediante el goal prepare-agent. El reporte HTML se genera en target/site/jacoco/index.html con el comando: `./mvnw test jacoco:report`

4.1 Cobertura consolidada (12 tests | Todos los integrantes)

Paquete	Instrucciones cubiertas	Cobertura	Ramas
ApuestaService	31 / 45 instr.	68%	n/a
ConfiguracionRiesgoService	30 / 35 instr.	85%	100%
EventoService	5 / 20 instr.	25%	n/a
MovimientoSaldoService	5 / 15 instr.	33%	n/a
UsuarioService	11 / 20 instr.	55%	n/a
MercadoService	0 / 20 instr.	0%	n/a
TOTAL service	82 / 155 instr.	52%	100%

5. Configuración JaCoCo en pom.xml

```
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.12</version>
  <executions>
    <execution>
      <goals>
        <goal>prepare-agent</goal>
      </goals>
    </execution>
  </executions>
</plugin>
```

El goal `prepare-agent` instrumenta automáticamente las clases durante la fase `initialize` de Maven. Al ejecutar `./mvnw test jacoco:report` se genera `target/jacoco.exec` y luego el reporte HTML.

6. Conclusiones

- Las 12 pruebas unitarias (3 por integrante) validan comportamiento esencial de la capa de servicios sin dependencia de la base de datos, alcanzando un **52% de cobertura de instrucciones** en el paquete `service` y **100% de cobertura de ramas**.
- El servicio con mayor cobertura es `ConfiguracionRiesgoService` con **85% de instrucciones y 100% de ramas**, gracias al patrón `upsert` que expone ramas condicionales claras.
- El uso de `@Mock` + `@InjectMocks` garantiza pruebas deterministas y rápidas en cualquier entorno, sin necesidad de levantar el contexto de Spring ni conectarse a PostgreSQL.
- `MercadoService` quedó con 0% de cobertura al no ser asignado a ningún integrante, lo que representa una oportunidad de mejora en futuras iteraciones.
- JaCoCo integrado con Maven permite ejecutar `./mvnw test jacoco:report` desde cualquier entorno y obtener evidencia objetiva y reproducible de la cobertura.
- .